



SERVICIO NACIONAL DE APRENDIZAJE – SENA

Informe de Codificación de Módulos Web

Evidencia: GA7-220501096-AA2-EV02 - Módulos de software codificados y probados

Proyecto: SAAAF - Sistema de Administración de Activos Fijos

Aprendiz: Brayan Giraldo García

Programa: Análisis y Desarrollo de Software

Fecha: 2026



1. Introducción

El presente informe técnico documenta la codificación del módulo web de Gestión de Activos Fijos del sistema SAAAF. La solución se construye con formularios HTML, JSP, Servlets, conexión JDBC a MySQL y operaciones CRUD, siguiendo los estándares de codificación definidos en evidencias anteriores y utilizando herramientas de versionamiento para el control de cambios del proyecto.

2. Objetivo

Codificar y probar un módulo web del sistema SAAAF usando Java Servlets, JSP, formularios HTML, métodos GET y POST, conexión JDBC a base de datos y herramientas de versionamiento, cumpliendo los requerimientos de la evidencia GA7-220501096-AA2-EV02.

3. Alcance del módulo codificado

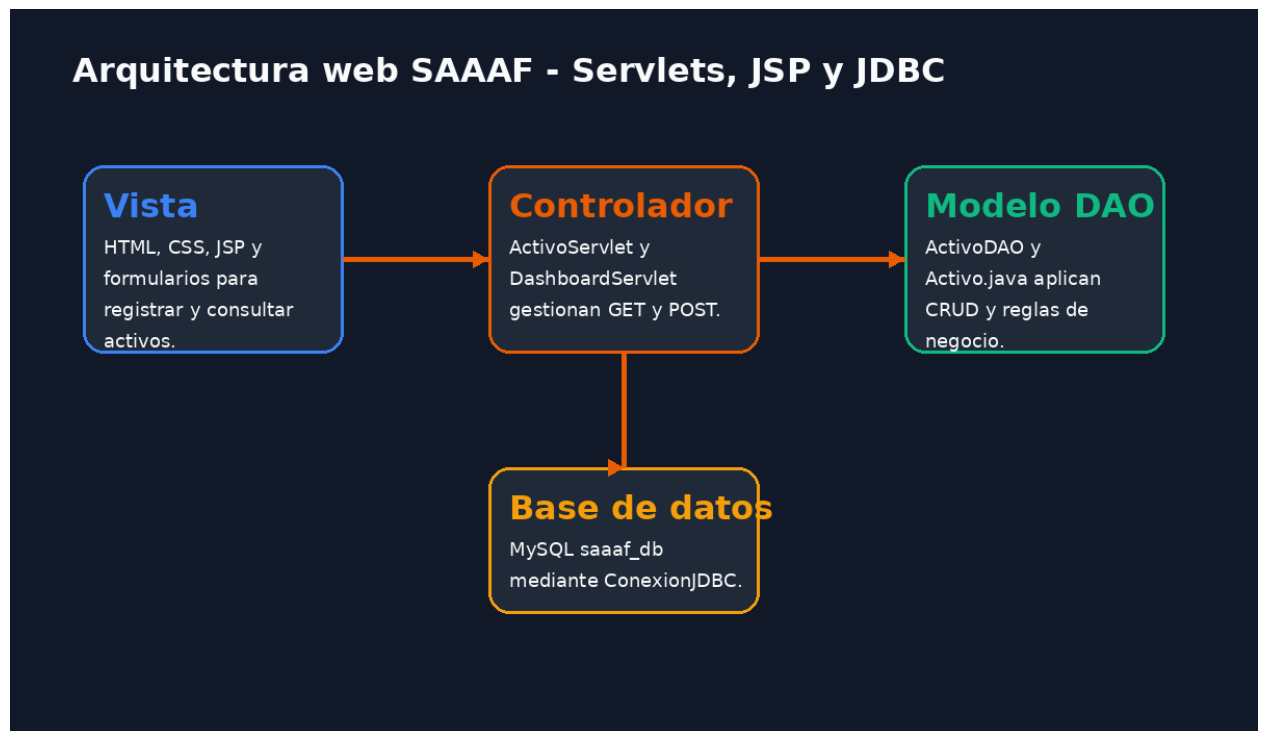
Elemento	Descripción
Módulo	Gestión de Activos Fijos del sistema SAAAF.
Funcionalidades	Listar, registrar, editar, eliminar y consultar indicadores de activos.
Tecnologías	Java, Servlets, JSP, JDBC, MySQL, HTML, CSS, JavaScript, Tomcat y Git.
Requerimiento principal	Administrar activos fijos, consultar su estado y conservar trazabilidad de la información.

4. Artefactos del ciclo de software considerados

Artefacto	Aplicación en esta evidencia
Historias de usuario	Se implementa la historia: como administrador deseo registrar y consultar activos para controlar el inventario.
Casos de uso	Registrar activo, listar activos, editar activo, eliminar activo y consultar dashboard.
Diseños y prototipos	Se conserva la estructura visual del proyecto SAAAF: tarjetas, paneles, colores por estado y navegación clara.
Informe técnico de plan de trabajo	Se aplican herramientas de desarrollo y versionamiento definidas previamente: Java, MySQL, Tomcat, Git y GitHub.
Estándares de codificación	Se aplican nombres de paquetes en minúscula, clases en PascalCase y métodos/variables en camelCase.



5. Arquitectura implementada



La arquitectura separa la vista, el controlador, el acceso a datos y la base de datos. Los formularios HTML/JSP envían solicitudes al servlet; el servlet procesa los parámetros y llama al DAO; el DAO usa JDBC para ejecutar operaciones en MySQL.

6. Estructura del proyecto

La carpeta comprimida contiene el proyecto web y los archivos de soporte requeridos para la entrega:

```
BrayanGiraldo_AA2_EV02/  
Proyecto_SAAAF_Web_Servlets/  
src/main/java/co/sena/saaaf/config/ConexionJDBC.java  
src/main/java/co/sena/saaaf/model/Activo.java  
src/main/java/co/sena/saaaf/dao/ActivoDAO.java  
src/main/java/co/sena/saaaf/servlet/ActivoServlet.java  
src/main/java/co/sena/saaaf/servlet/DashboardServlet.java  
src/main/webapp/index.html  
src/main/webapp/registro-activo.html  
src/main/webapp/WEB-INF/views/activos.jsp  
src/main/webapp/WEB-INF/views/form-activo.jsp  
src/main/webapp/WEB-INF/views/dashboard.jsp  
database/saaaf_web.sql  
git_comandos.txt  
enlace_repositorio.txt
```

7. Cumplimiento de requisitos técnicos

Requisito de la guía	Implementación realizada
Formularios HTML con servlets	Se incluye registro-activo.html con formulario POST hacia



	el servlet /activos. También se usa index.html con formulario GET.
Métodos GET y POST	ActivoServlet implementa doGet para listar, editar y eliminar; doPost para insertar y actualizar activos.
Elementos JSP	Se implementan activos.jsp, form-activo.jsp y dashboard.jsp para vistas dinámicas.
Herramientas de versionamiento	Se incluye .gitignore, git_comandos.txt y enlace_repositorio.txt para registrar el repositorio remoto.

8. CRUD implementado

Operación	Método HTTP	Archivo responsable	Descripción
Crear	POST	ActivoServlet + ActivoDAO.insertarActivo()	Registra un nuevo activo desde formulario HTML/JSP.
Consultar	GET	ActivoServlet + ActivoDAO.listarActivos()	Lista activos registrados desde MySQL.
Actualizar	POST	ActivoServlet + ActivoDAO.actualizarActivo()	Actualiza los datos de un activo existente.
Eliminar	GET	ActivoServlet + ActivoDAO.eliminarActivo()	Elimina un activo a partir del parámetro id.

9. Conexión JDBC

La clase ConexionJDBC centraliza los parámetros de conexión a MySQL. El DAO utiliza PreparedStatement para reducir riesgos de inyección SQL y para mantener un acceso estructurado a la base de datos.

```
public static Connection obtenerConexion() throws SQLException {  
    Class.forName("com.mysql.cj.jdbc.Driver");  
    return DriverManager.getConnection(URL, USUARIO, CLAVE);  
}
```

10. Estándar de codificación aplicado

Elemento	Estándar aplicado	Ejemplo
Paquetes	Minúsculas y separados por dominio.	co.sena.saaaf.servlet
Clases	PascalCase.	ActivoServlet, ActivoDAO, ConexionJDBC
Métodos	camelCase y verbos descriptivos.	listarActivos(), insertarActivo(), obtenerConexion()
Variables	camelCase y significado funcional.	codigoActivo, nombreActivo, fechaRegistro
Constantes	Mayúsculas sostenidas con guion bajo.	SQL_LISTAR, SQL_INSERTAR



11. Pruebas realizadas

Prueba	Entrada	Resultado esperado
Listar activos	GET /activos	Se visualiza la tabla de activos cargada desde la base de datos.
Registrar activo	Formulario HTML POST /activos	Se inserta el activo en MySQL y se redirecciona al listado.
Editar activo	GET /activos?accion=editar&id=1	Se carga el formulario JSP con datos del activo seleccionado.
Eliminar activo	GET /activos?accion=eliminar&id=1	Se elimina el registro y se actualiza el listado.
Dashboard	GET /dashboard	Se muestran indicadores total, disponibles, asignados y mantenimiento.

12. Versionamiento del proyecto

El proyecto incluye archivos para control de versiones. El aprendiz debe crear un repositorio remoto y reemplazar el enlace de ejemplo incluido en enlace_repositorio.txt por el enlace real.

```
git init
git add .
git commit -m "Estructura inicial proyecto web SAAAF con Servlets JSP y JDBC"
git branch -M main
git remote add origin https://github.com/USUARIO/SAAAF-Web-Servlets.git
git push -u origin main
```

13. Tabla de cumplimiento

Indicador de logro	Cumplimiento en la solución	Estado
El código contiene formularios HTML con servlets.	Se implementan index.html y registro-activo.html conectados al servlet /activos.	Cumple
Utiliza los métodos GET y POST para uso de parámetros.	ActivoServlet implementa doGet y doPost usando request.getParameter().	Cumple
Implementa elementos de JSP.	Se implementan vistas JSP para dashboard, listado y formulario de activos.	Cumple
Utiliza herramientas para versionamiento del código.	Se entregan .gitignore, comandos Git y archivo para enlace del repositorio.	Cumple

14. Conclusiones

- El módulo web de Gestión de Activos de SAAAF cumple con el enfoque solicitado de formularios HTML, Servlets, JSP y JDBC.
- La solución implementa operaciones CRUD completas para registrar, consultar, actualizar y eliminar activos fijos.



- El proyecto mantiene estándares de codificación definidos y una estructura organizada por paquetes.
- La evidencia queda preparada para ser versionada en Git y publicada en un repositorio remoto.

15. Referencias

- Instrumento de evaluación GA7-220501096-AA2-EV02.
- Proyecto SAAAF - Sistema de Administración de Activos Fijos.
- Material de formación SENA sobre Java, Servlets, JSP, JDBC y versionamiento Git.